Benchmarking DAOS Filesystem on Aurora

Rebecca George [1,2] Xinxin Mei [2] Jie Ren [1]

[1]William & Mary [2]Jefferson Lab

DAOS Storage System

Distributed Asynchronous Object Storage (DAOS) is a high performance storage system using key-array-value store allowing end-to-end user space operation, asynchronous I/O operation, and non-blocking I/O.

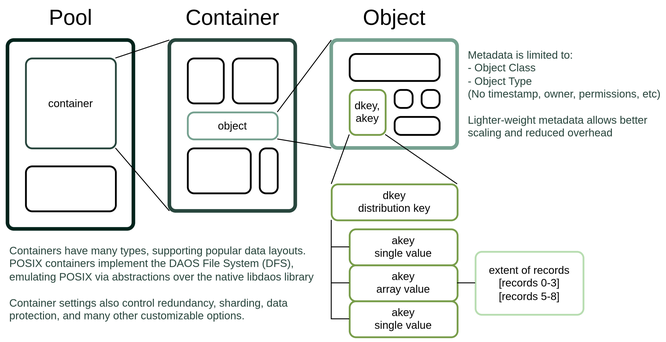

Figure 1. DAOS pool data hierarchy (adapted from [2])

DAOS is provisioned in pools (storage allocations) wherein containers, which are logical collections of objects (key-value stores), may be created. Objects' keys have a distribution key (dkey, determining the storage target) and attribute key (akey), with values of either single atomic values or arrays.

DAOS can be accessed via the native libdaos library, libpil4dfs interception library (intercepts I/O and metadata), libioil interception library (intercepts I/O only), and mounting a container via DFuse.

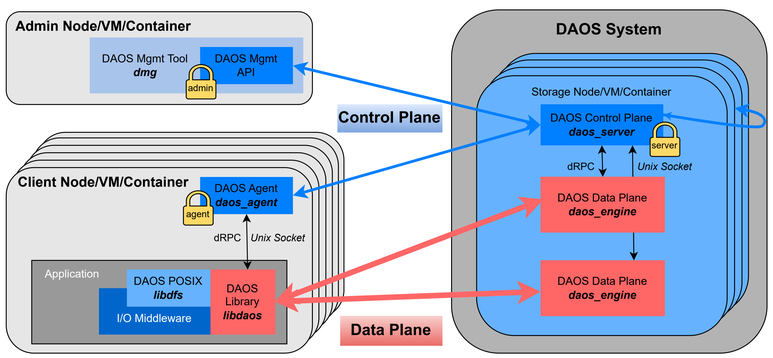

Figure 2. DAOS client, storage, and administrative node communication. [1] Applications on client nodes send data to/from DAOS storage nodes. Administrators may use the dmg tool and management API to communicate with the DAOS server (storage-side) and agent (client-side).

Aurora

- Argonne National Lab's Aurora supercomputer has 10,624 compute nodes with 230 PB of DAOS storage on 1024 DAOS server nodes.
- Aurora compute nodes have 8 HPE Slingshot-11 NICs, using dragonfly topology and use two Intel Xeon CPU Max Series, with 52 cores each. For IOR and IO500 we bind to 48 cores per node.
- Our pool had 127 DAOS server nodes (4064 targets, 32 targets per node) with 3 TB Tier 0 (SCM) storage and 97 TB Tier 1 (NVMe) storage.

Benchmarking

We evaluated this storage system using the fio, IOR, mdtest, and IO500 I/O benchmarks. We aim to characterize the I/O and metadata performance bottlenecks under a variety of access patterns and client configurations on a production scale system.

IO500

IO500 is a standard high-performance computing benchmarking suite, evaluating both I/O (IOR) and metadata (mdtest) operations.

Test	DFS API	POSIX API
ior-easy-write	92.2 GiB/s	81.3 GiB/s
ior-easy-read	212.6 GiB/s	10.8 GiB/s
ior-hard-write	3.8 GiB/s	0.7 GiB/s
ior-hard-read	15.3 GiB/s	3.2 GiB/s
Bandwidth Score	32.6 GiB/s	6.5 GiB/s

Table 1. 8 node IO500 bandwidth

Test	DFS API	POSIX API
find	775.6 kIOPS	1.6 kIOPS
mdtest-easy-write	219.4 kIOPS	4.0 kIOPS
mdtest-hard-write	60.6 kIOPS	1.1 kIOPS
mdtest-hard-read	807.3 kIOPS	3.9 kIOPS
mdtest-easy-stat	1112.2 kIOPS	4.0 kIOPS
mdtest-hard-stat	1098.3 kIOPS	3.9 kIOPS
mdtest-easy-delete	301.0 kIOPS	3.4 kIOPS
mdtest-hard-delete	302.6 kIOPS	0.4 kIOPS
IOPS Score	417.7 kIOPS	2.2 kIOPS

Table 2. 8 node IO500 IOPS

DFS API achieved ≈5x higher bandwidths and ≈190x higher IOPS than POSIX API on a DFuse mounted POSIX container, making major improvements to metadata and reading operations

DFuse POSIX Bandwidth

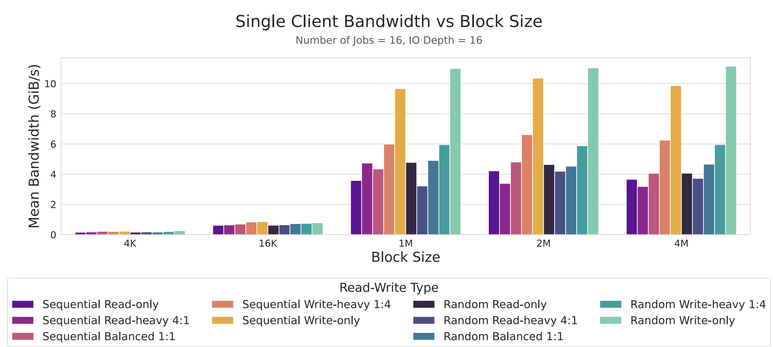

Figure 3. Single client bandwidth vs block size (I/O unit size) for fio using pvsync I/O engine.

- DFuse POSIX bandwidth increased with I/O unit size, saturating at I/O unit size of 1-2 MiB
- Workloads with more writing, relative to reading, achieved higher throughput via DFuse
- Random vs sequential access had little effect on throughput

IOR Latency

Figure 4. For a transfer size of 1 MiB, the IOR latency vs number of tasks, using DFS API.

DFS I/O latency increased with more tasks and nodes, with greater effects on writing than reading

IOR Bandwidth

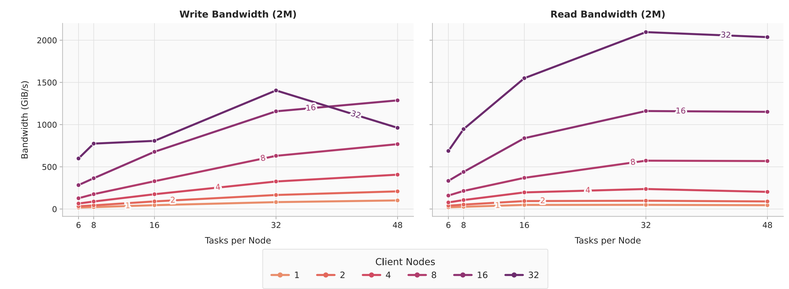

Figure 5. For a transfer size (I/O unit size) of 2 MiB, the IOR bandwidth vs number of tasks, using the DFS API

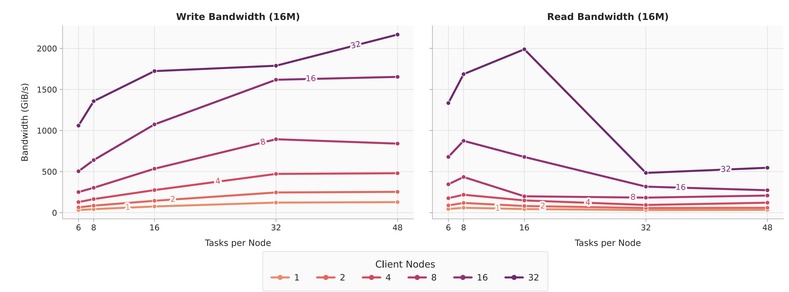

Figure 6. For a transfer size (I/O unit size) of 16 MiB, the IOR bandwidth vs number of tasks, using the DFS API.

- Read and write bandwidth increased with as tasks increased, saturating around 32 tasks
- At high task counts, sufficiently large transfer sizes reduce read bandwidth

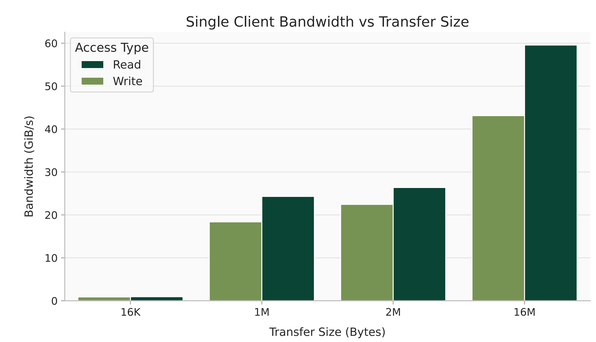

Figure 7. For 8 tasks per node, the IOR bandwidth vs number of tasks, using the DFS API.

- For a single client, reading is slightly more performant than writing across transfer sizes
- Sufficiently small transfer sizes (16 KiB) provide low throughput, both for a single client (Figure 7) and all numbers of nodes and tasks per node tested

Acknowledgments

This research used resources of the Argonne Leadership Computing Facility, which is a U.S. Department of Energy Office of Science User Facility operated under contract DE-AC02-06CH11357. This work was supported by NSF OAC-2348350.

References

[1] Xinxin Mei and et al. Exploring daos as a burst buffer for a 100 gbps daq real-time streaming system, November 2024.

[2] DAOS Project. Welcome to daos version 2.6! - daos v2.6, 2025. URL `https://docs.daos.io/latest/`.

rcgeorge@wm.edu